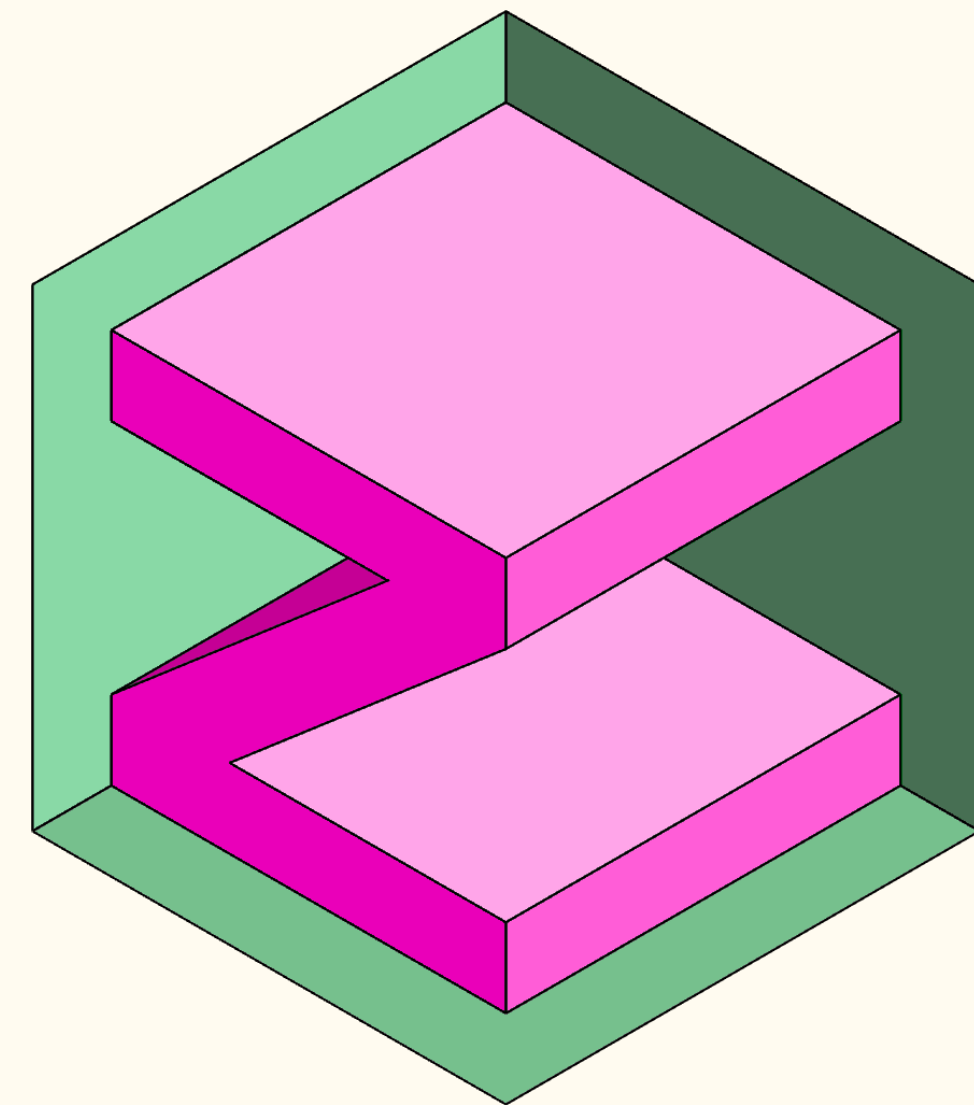


Self-Updating Go Binaries

Distributed via Container Registries





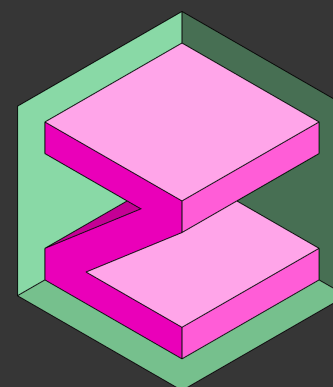
About me

Christian Blättler

zeitlos.software
Mattenhofstrasse 5
3007 Bern

<https://zeitlos.software>

zeitlos



flex.plane

On-prem virtualization platform for containers & VMs

flex.plane

JL

Tenant

Dev Tenant

Tenant context switching coming soon

Dashboard

Clusters

Nodes

Virtual Machines

Networks

Load Balancers

Kubernetes

Settings

Virtual Machines

Create VM

Machine	Resources	Networks	Uptime	Status	Actions
flexkube-test-001-system-0 flxp-sandbox-pve-01	2 CPUs 4 GiB	netf2bf7 10.0.0.10	an hour	running	
flexkube-test-001-system-1 flxp-sandbox-pve-01	2 CPUs 4 GiB	netf2bf7 10.0.0.11	an hour	running	
ubuntu10 flxp-sandbox-pve-02	1 CPUs 1 GiB	staging 10.10.10.10	an hour	running	

Displaying 3 out of 3 results

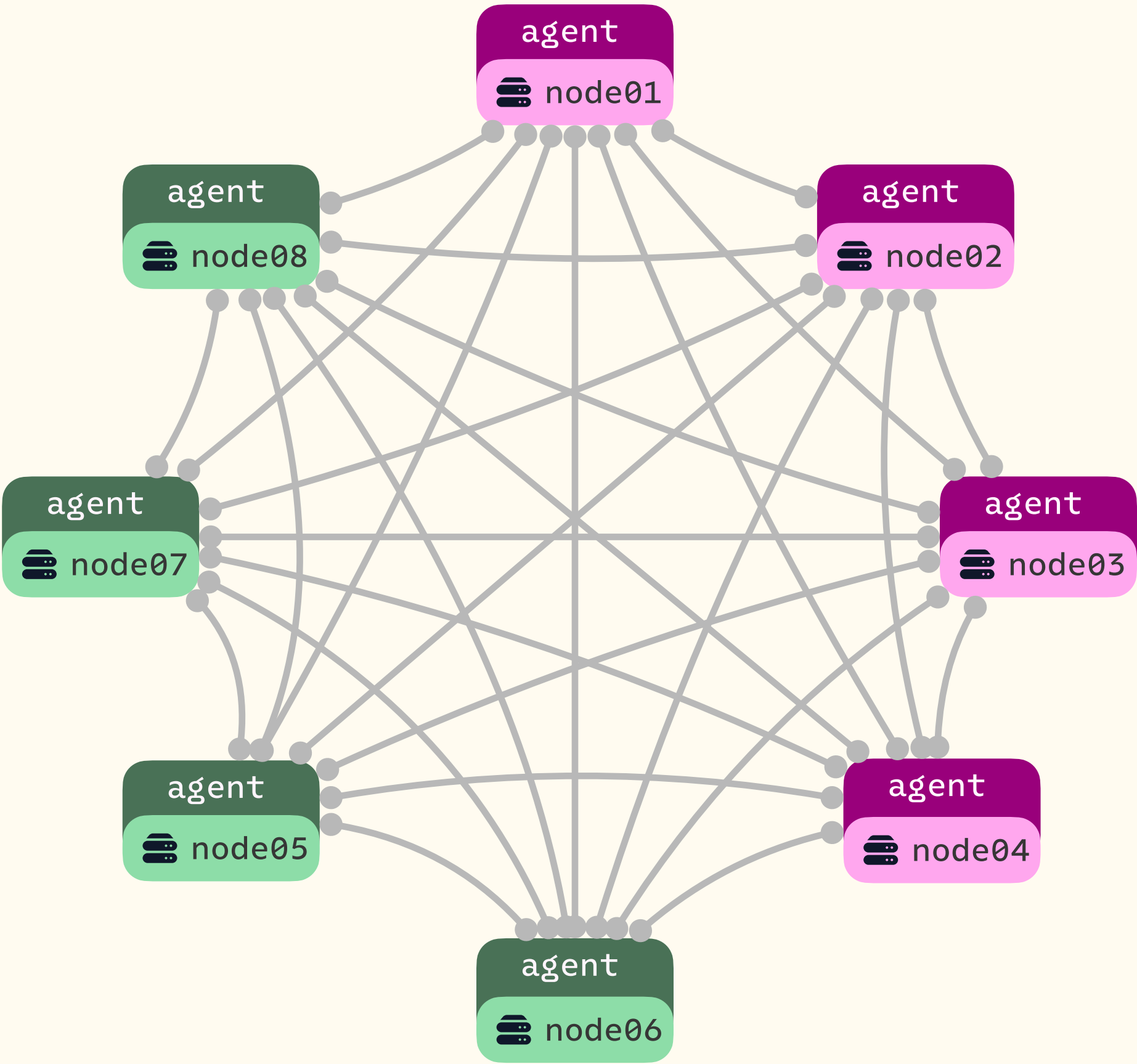
About VMs

Virtual Machines are flexible, virtualized servers that can be created within seconds and run application workloads. Snapshots allow you to restore the VM to a desired state in the past while backups ensure that you don't loose any critical data.



flex.plane Agent

Runs on every node



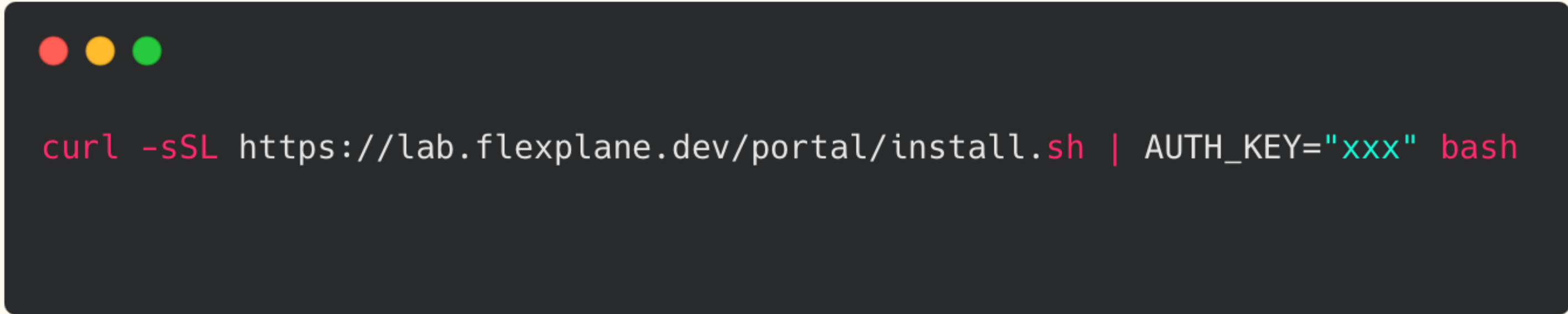
[● ◀]
systemd

gRPC



Where we come from

...any why this sucks



```
curl -sSL https://lab.flexplane.dev/portal/install.sh | AUTH_KEY="xxx" bash
```

Distribution

- Only latest version
- Wasted bandwidth

Coordination

- Uncoordinated
- Time-consuming

Update Process

- Fragile
- No rollback

Design Goals

(I learned at university this is how proper engineers do it)

Distribution

- Versionized
- Cached
- Globally distributed

Coordination

- Centrally managed
- Automated

Update Process

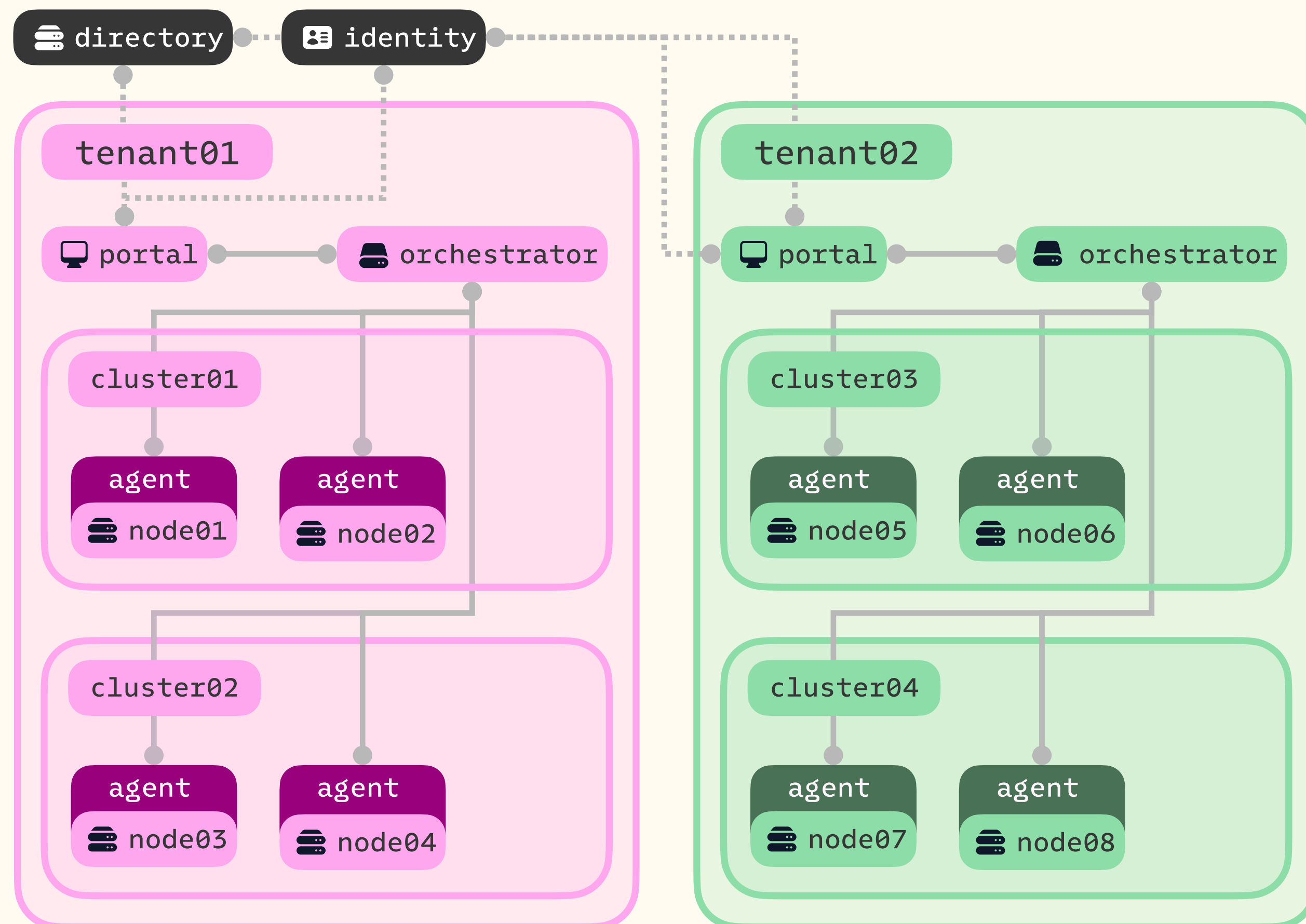
- Resilient
- Minimal downtime

Distribution

The solution was right under our nose



Coordination



- Extend Agent gRPC API
- Central Management via Portal
- Custom update schedules

Solution

Self-contained update mechanism

- Go Library
- Wrap the Agent process
- Download updates via ORAS Go SDK
- Atomic symlink swapping

Update Flow

Piecing together the puzzle

1. Agent triggers installation of update
2. Supervisor downloads update
3. Atomically swaps current symlink
4. Kills process
5. systemd restarts service (new version)

Rollbacks



```
/opt/testapp/  
|-- current -> /opt/testapp/versions/0.0.2  
|-- previous-20251105-131034 -> /opt/testapp/versions/0.0.1  
`-- versions  
    |-- 0.0.1  
    |    `-- testapp  
    `-- 0.0.2  
        `-- testapp
```

Usage

```
package main

import (
    "github.com/zeitlos/knockknock"
    "github.com/zeitlos/knockknock/config"
)

var Version = "0.0.1"

func main() {
    knockknock.Run(
        config.New("myapp").
            WithRepo("ghcr.io/myorg/myapp").
            WithVersion(Version),
        run,
    )
}

func run() {
    // Your application code here
}
```

```
// Checking for updates and listing available versions
update, versions, err := knockknock.
    Client().
    CheckForUpdate(r.Context())

if err != nil {
    slog.Error("failed to check for updates", "error", err)
}

// ...

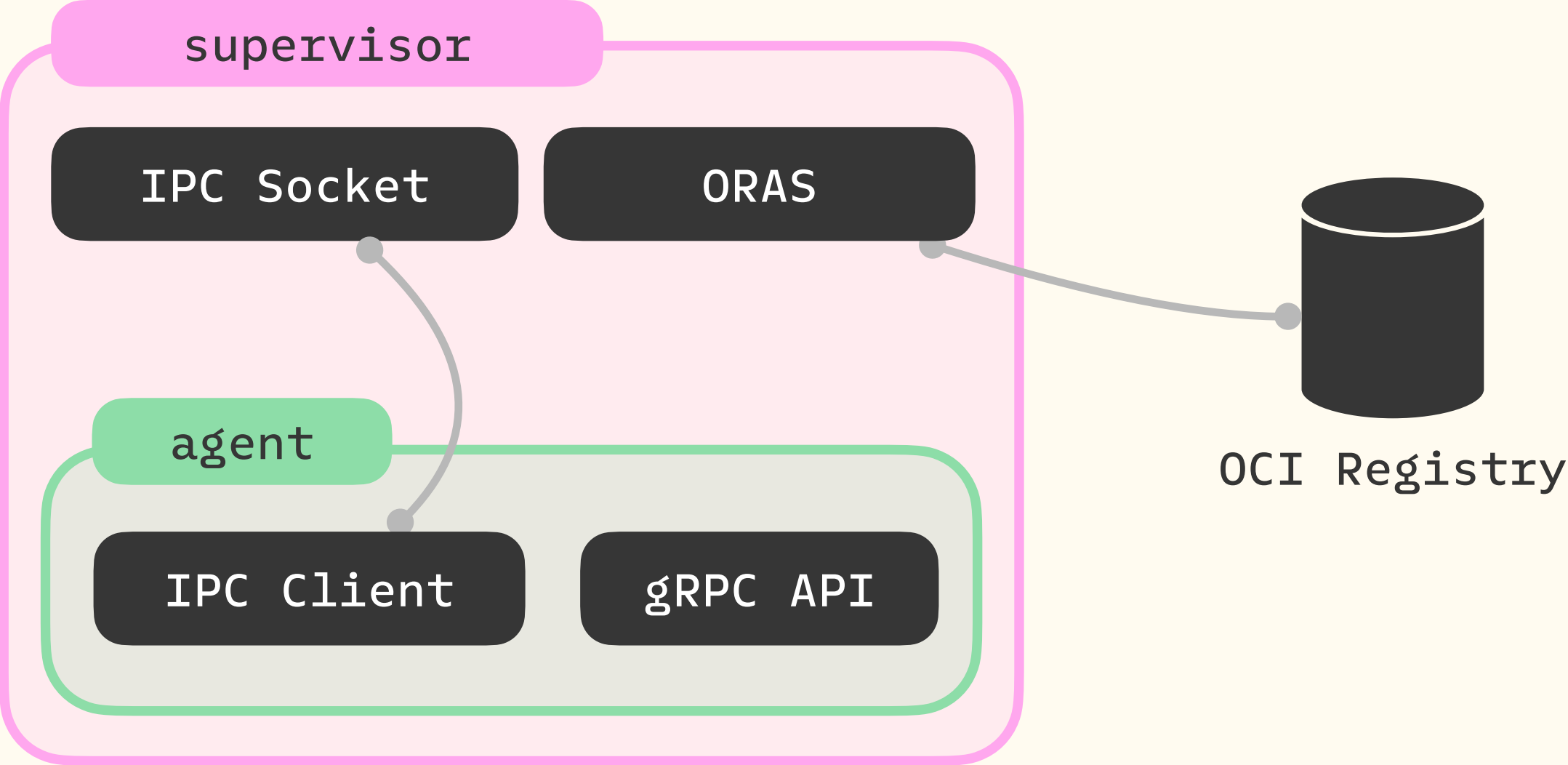
// Installing an update

err := knockknock.
    Client().
    Update(context.Background(), selectedVersion)

if err != nil {
    slog.Error("failed to update", "error", err)
}
```

Architecture

Process-ception



The dreaded Live-Demo

What could possibly go wrong...

Evaluating Design Goals

Distribution

- Versionized
- Cached
- Globally distributed

Coordination

- Centrally managed
- Automated

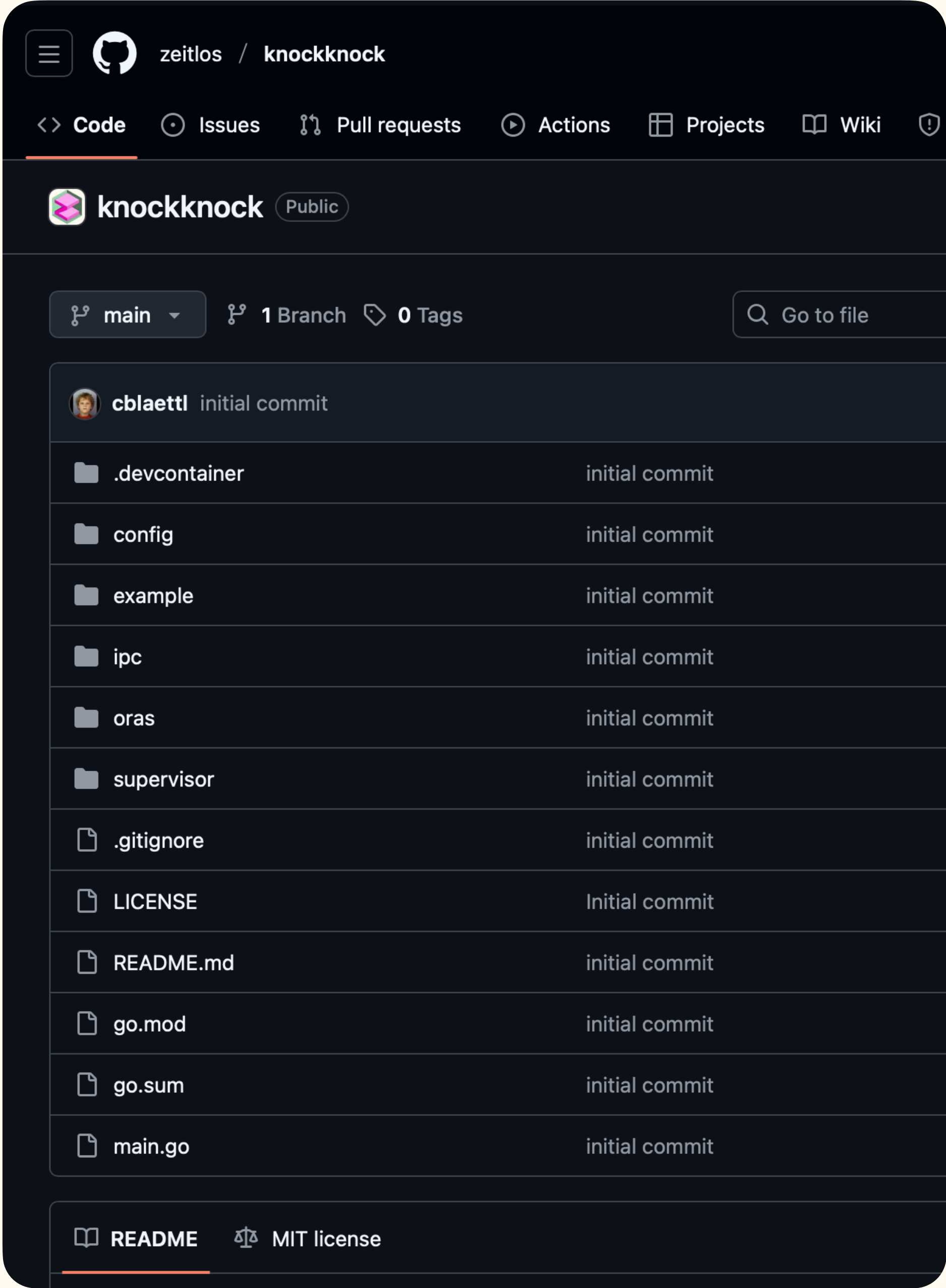
Update Process

- Resilient
- Minimal downtime



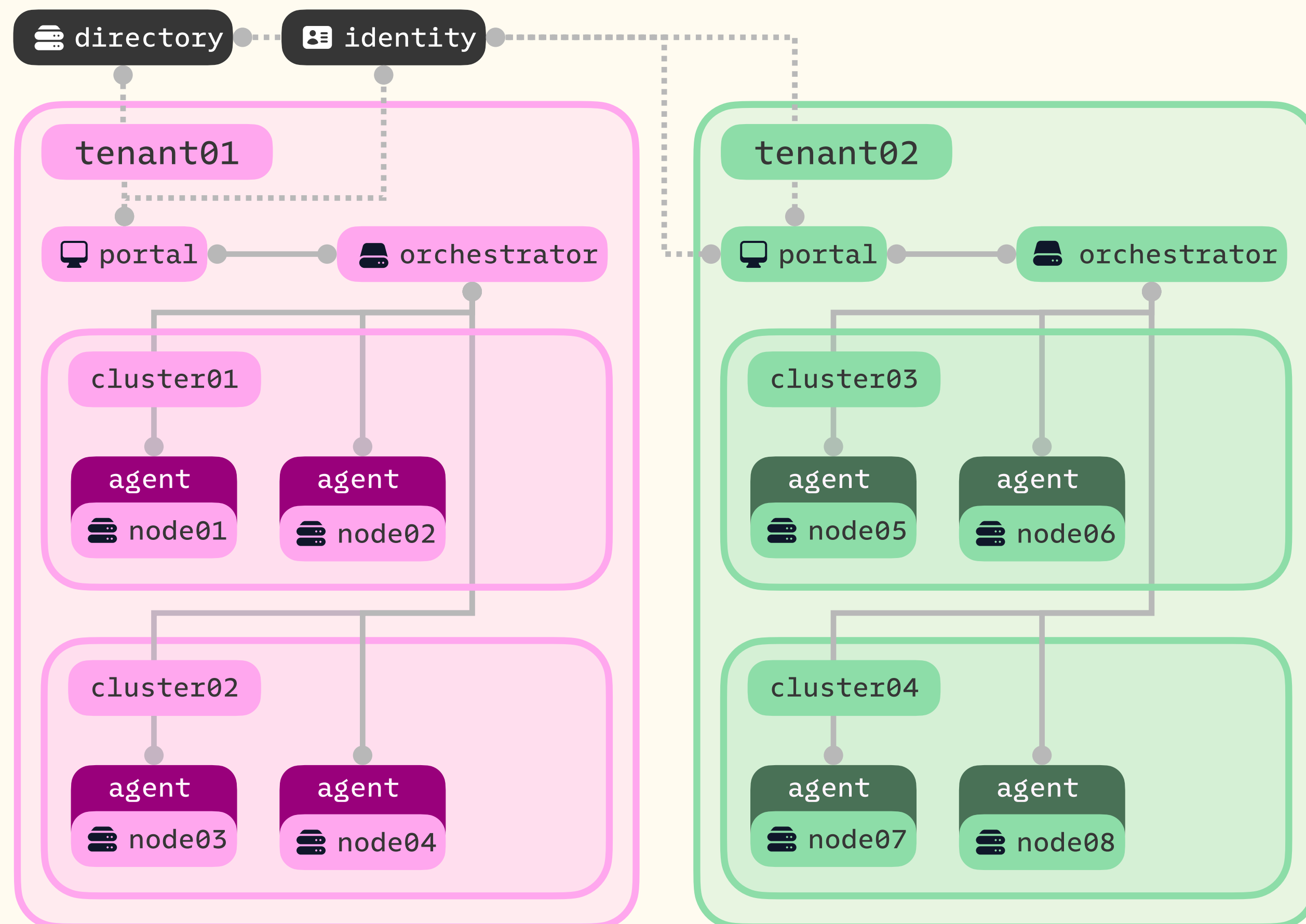
Thank you!

It's open source (MIT license)
github.com/zeitlos/knockknock



Extra Slides

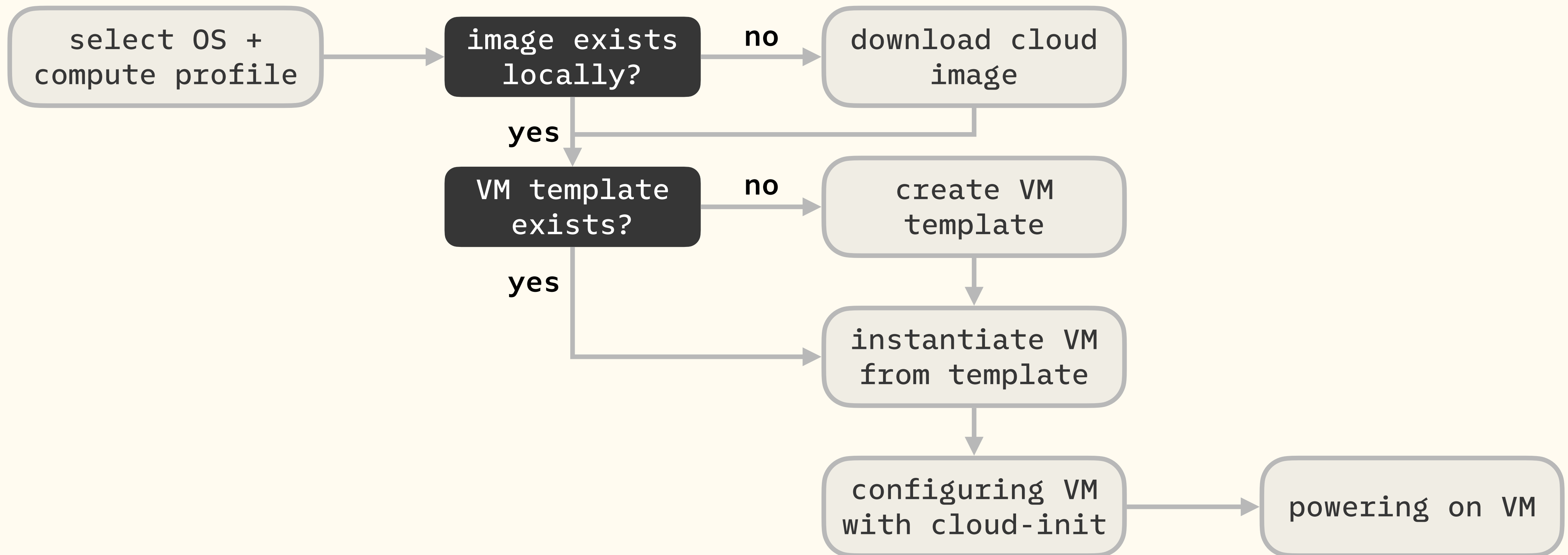
System Architecture



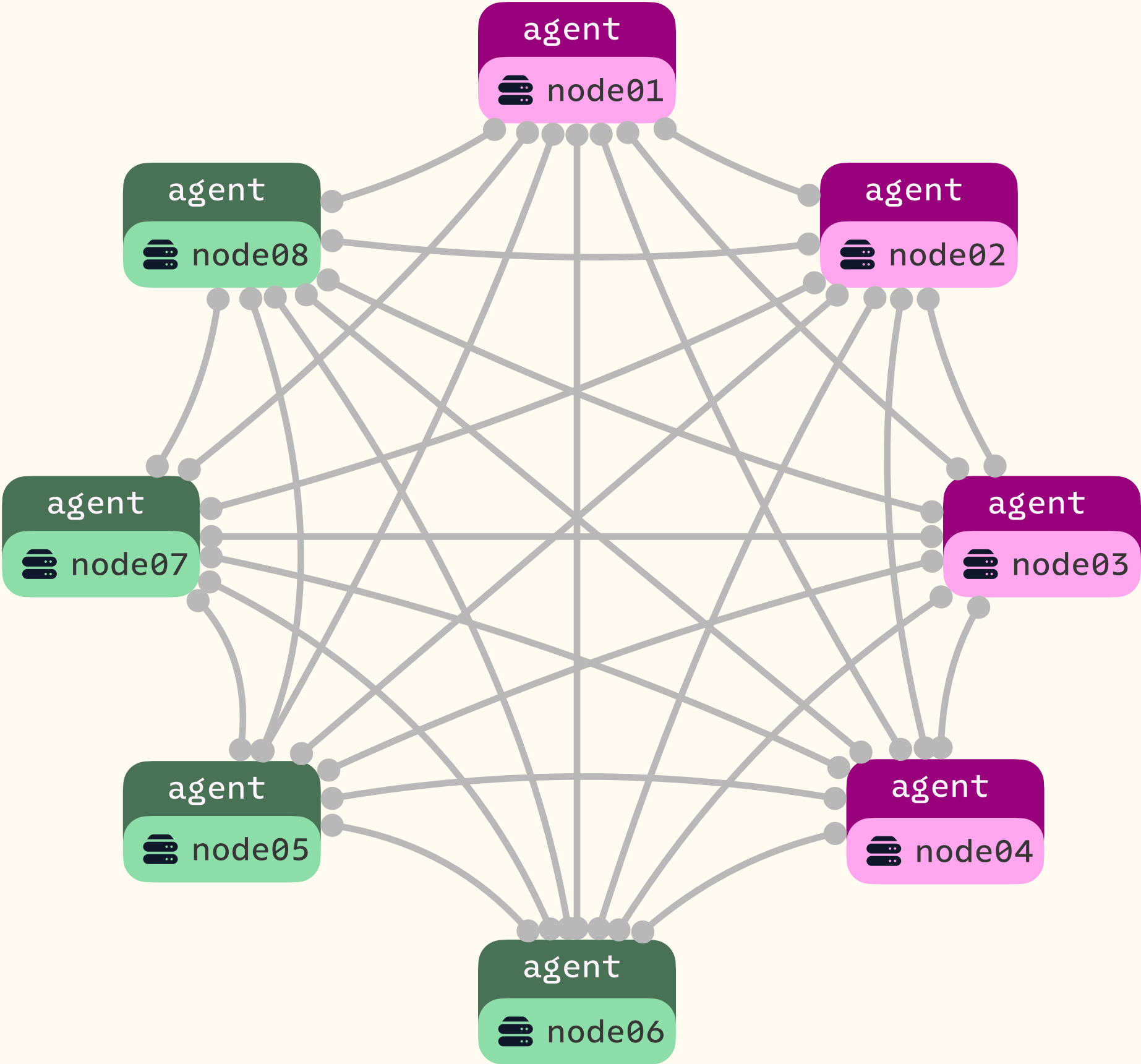
Design Goals

- Choose between managed or self-hostable control plane
- Isolated multi-tenancy
- Centralized user management
- No port-forwarding / firewall configuration required

Virtual Machine Provisioning



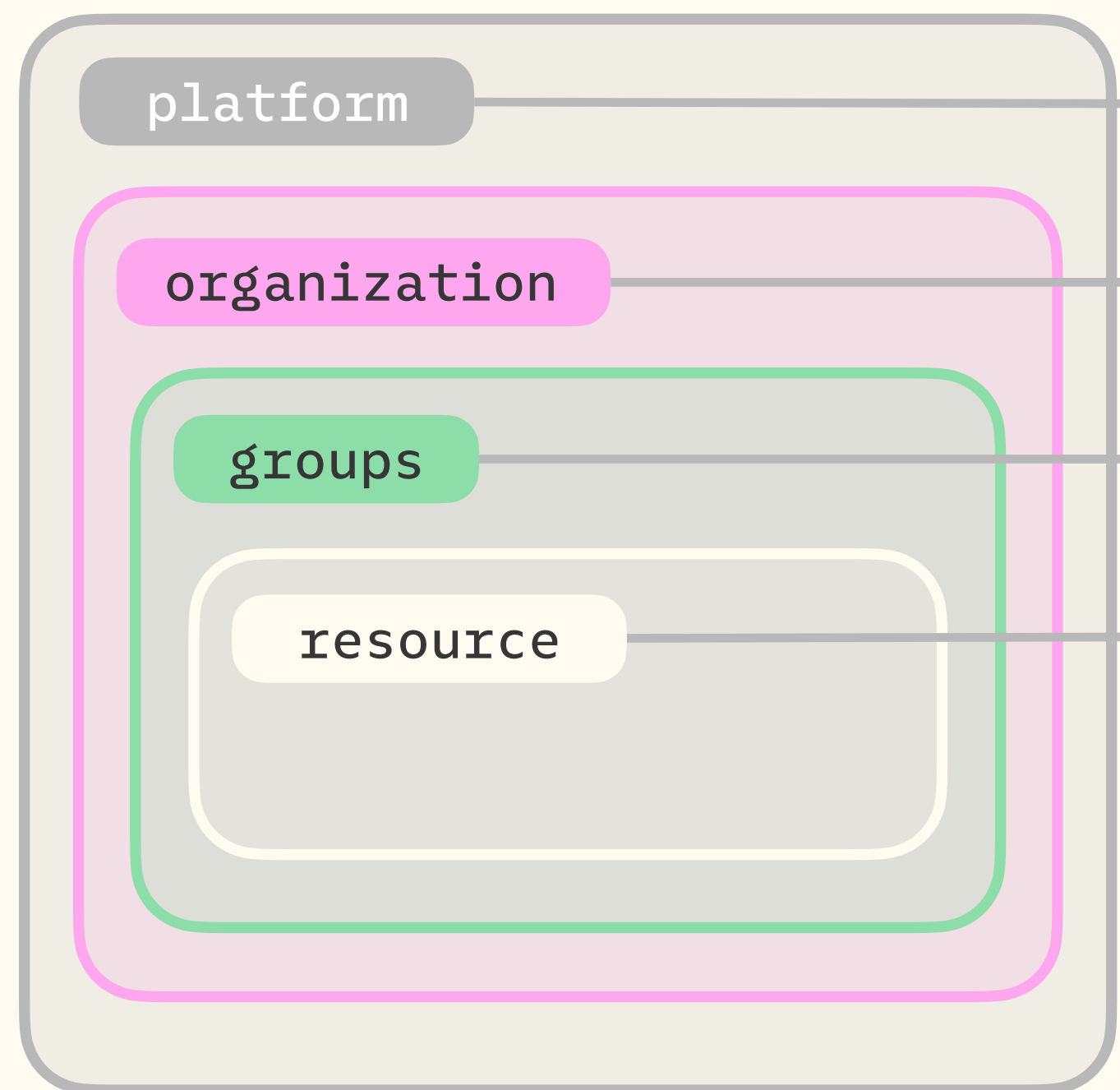
Multi-Cluster Management



Identity

Proxmox Equivalent

Use-cases



Authentication Realms

Zitadel, Authentik, ...

Cluster(s)

Company, Tenant, Homelab, ...

Resource Pool

Customer, Team, Project, ...

VM, Network

Service, Component, ...